

5) Optimize Hyperparameters and Network Architectures

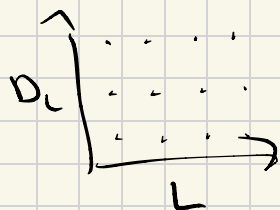
- choose hyperparameters: learning rate, momentum μ , weight decay
- architectural choices: number of layers L , number of neurons per layer, act. func.
- Goal: improved generalization
- Pilot experiments:

Test our candidate settings on:

- a) reduced data training set
- b) few iterations of optimization
- c) possibly use fewer features

• Hyperparameter Search Methods:

- Grid search: Try combinations from a predefined grid



- Random search: Sample values from prob. distributions

- Bayesian optimization

- let λ denote hyperparameters, and x architectural params:

you want to model the validation loss
to guide search

bi-level optimization:

1) inner loop

$$\theta^*(\lambda, x) = \arg\min_{\theta} \mathcal{L}(\theta; \lambda, x)$$

2) outer loop:

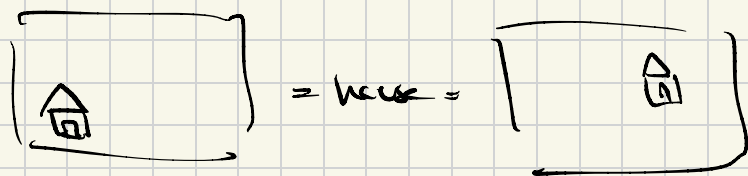
$$f(\lambda, x) = \mathcal{L}_{val}(\theta^*(\lambda, x))$$

then try to

$$\min_{\lambda, x} f(\lambda, x)$$

2.14 Introduction to Convolutional Neural Networks (CNNs)

- typically used for images
- weight sharing $\Rightarrow$ reduces the number of params.
- key insight: objects in an image do not change when they move location



$\Rightarrow$ requirement: "translation equivariance"

2.14.1 How to Achieve Translation Equivariance

- use specific filters / conv. kernels
- for a 1D function $f(t)$, we can write
$$y(t) = \text{Filter}(f(t))$$

• Translation Equivariance Requirement:

If we shift $f(t)$ by some t_0 , i.e. $f(t+t_0)$, then, if we define the translation operator:

$$(T_{t_0} f)(t) = f(t+t_0)$$

then the condition for equivariance is:

$$\text{Filter}[T_{t_0} f] = T_{t_0}(\text{Filter}[f])$$

$\forall f, \forall t_0$.

• Theorem:

Any linear, shift invariant filter can be written as a convolution:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(t') g(t-t') dt'$$

• translated to discrete setting:

$$(f * g)(n) = \sum_{k=-\infty}^{\infty} \underbrace{f[k]}_{\text{signal}} \underbrace{g(n-k)}_{\text{filter / kernel}}$$

- Translate to 2D:

$$(f * g)[m, n] = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} f[i, j] g[m-i, n-j]$$

Note: classical def. does not support "strides", ^{e.g. image} in practice we treat local patches of images as independent, and only convolve at "s-th" position)

2.14.2 Basic Building Blocks of CNNs

How to build a filter for image processing?

- A filter will be implemented by a matrix of numbers

- For example:

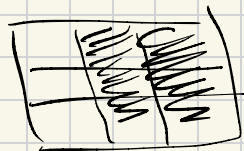
$$K = \begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix}$$



→ this filter is designed to detect vertical edges

- input image $I \in \mathbb{R}^{3 \times 3}$

$$I = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$



- Step-by-step convolution

(i) top-left patch:

$$\begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix} \Rightarrow (-1) \cdot 0 + 1 \cdot 1 + -1 \cdot 0 + 1 \cdot 1 = 2$$

(ii) top-middle patch

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \Rightarrow (-1) \cdot 1 + 1 \cdot 1 + -1 \cdot 1 + 1 \cdot 1 = 0$$

(iii) bottom left

$$\begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix} \Rightarrow 2$$

(iv) bottom middle $\Rightarrow 0$

$$I * K = \begin{bmatrix} 2 & 0 \\ 2 & 0 \end{bmatrix}$$

• What is a stride? s

stride s determines how many pixels the filter moves ("slides") at each step when scanning the input image
 $\rightarrow$ affects output size

• What is padding p ?

• Padding involves adding extra pixels (usually

0's) around the border

$$H \cdot W \xrightarrow{p=1} (H+2) \times (W+2)$$

1) Convolutional layer

- say we have image $I \in \mathbb{R}^{H \times W}$ and a filter $K \in \mathbb{R}^{k \times k}$
- introduce padding: $I_{\text{pad}} \in \mathbb{R}^{(H+2) \times (W+2)}$
- define stride s
- Element-wise multiplication

for each (i, j) , compute the convolution:

$$O(i, j) = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} I_{\text{pad}}(i \cdot s + u, j \cdot s + v) \underbrace{K(u, v)}_{\text{kernel element}}$$

output image = "feature maps"

2) Nonlinearity

- let the input be a 3-D tensor:
 $x \in \mathbb{R}^{H \times W \times C_{\text{in}}}$

- let the filter bank tensor be:

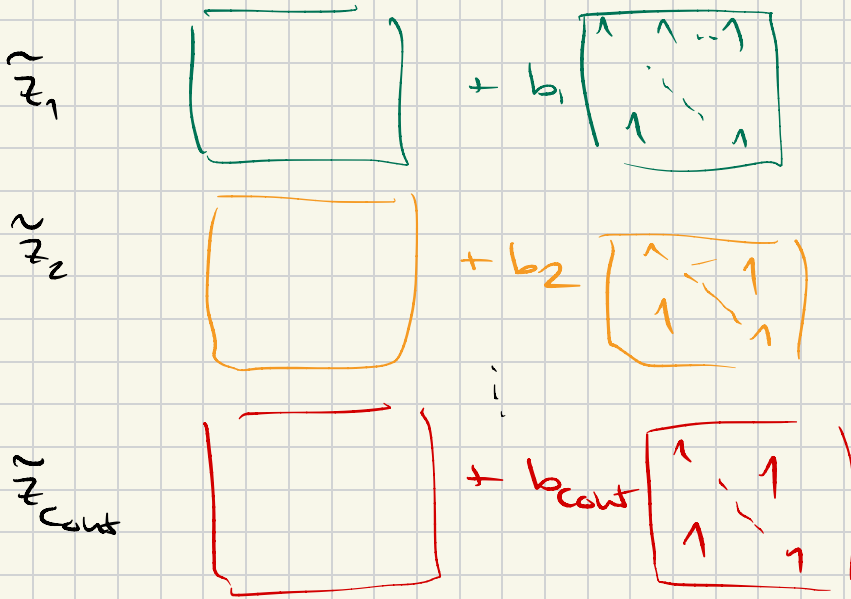
$$K \in \mathbb{R}^{k_H \times k_W \times C_{\text{in}} \times C_{\text{out}}}$$

$\underbrace{\quad}_{\text{no. of input images / feature maps}} \quad \underbrace{\quad}_{\text{number of output feature maps}}$

- The convolution operation:

$$\tilde{z} = \text{conv}(x; K, s, p) + b \in \mathbb{R}^{\# \times w' \times c_{out}}$$

Output feature maps

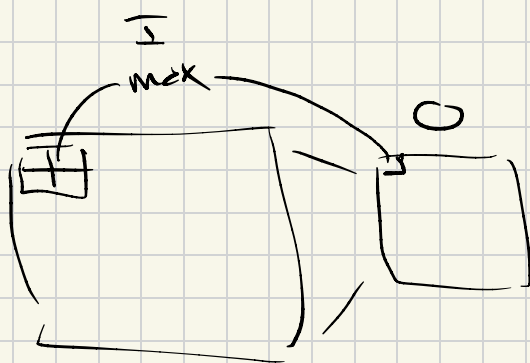


→ adding b "channel-wise", scalar per feature map

- Apply $z = \phi(\tilde{z}) \in \mathbb{R}^{\# \times w' \times c_{out}}$ elementwise.

3) Pooling layers

- Reduce the spatial dimension of the feature maps



3.1) Max pooling:

say we have some window W

$$O(i,j) = \max_{(u,v) \in W} I(u,v)$$

3.2) Average pooling:

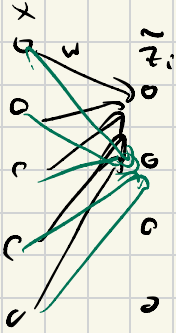
$$O(i,j) = \frac{1}{|W|} \sum_{(u,v) \in W} I(u,v)$$

4) Fully connected layers

- From the final pooling layer, we obtain a set of feature maps; these we "flatten" into a single vector $x(\# \times W \times D) \rightarrow 1 \text{ vector} \in \mathbb{R}^{(x \times W \times D)}$

↑
feature maps

- compute pre-activation



$$\tilde{z}_i = x^T w$$

- compute activation:

$$z_i = \phi(\tilde{z}_i)$$

- end up with $z = \begin{bmatrix} z_1 \\ \vdots \\ z_{D_z} \end{bmatrix}$, pass through

softmax:

$$\hat{y}_i = \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1} e^{z_j}}$$

Summary: Forward Pass in a CNN

- Input: Training instance (x_i, y_i) ; learning rate τ ;
parameters $\Theta_{\text{CNN}} = \{K^{(l)}, b^{(l)}\}_{l=1}^{L_C}$ for the conv.

parts, $\Theta_{\text{FC}} = \{W^{(m)}, b^{(m)}\}_{m=1}^{L_{FC}}$

- Output: $\Theta_{\text{CNN}}, \Theta_{\text{FC}}$

- Forward pass:

$$\text{Set } z^{(0)} \leftarrow x$$

for $l = 1$ to L_c do

- convolution: (prediction)

$$\tilde{z}^{(l)} \leftarrow \text{conv}(z^{(l-1)}; K^{(l)}, s^{(l)}, p^{(l)}) + b^{(l)}$$

- nonlinearity: (activation)

$$z^{(l)} \leftarrow \phi(\tilde{z}^{(l)}) \quad (\text{element-wise})$$

- optionally: pooling

$$z^{(l)} \leftarrow \text{pool}(z^{(l)})$$

end for

Flatten z^{L_c} into a vector f .

$$\text{set } a^{(0)} \leftarrow f$$

for $m = 1$ to $L_{fc} - 1$ do

- compute prediction

$$a^{(m)} \leftarrow W^{(m)} a^{(m-1)} + b^{(m)}$$

- Apply nonlinearity

$$z^{(m)} \leftarrow \phi(a^{(m)})$$

end for

Compute final output preactivation

$$\tilde{z}^{L_C} \leftarrow W^{(L_C)} a^{(L_C-1)} + b^{(L_C)}$$

Compute softmax

$$\hat{y} = p \leftarrow \text{softmax}(\tilde{z}^{L_C})$$

Compute the loss:

$$\mathcal{L} \leftarrow - \sum_i y_i \ln(p_i)$$

2.14.3 Data Augmentation

- Motivation: CNNs are not invariant to transformations such as rotation, scaling, changes in illumination
- Approach: Augment data; strategy to artificially increase your data and diversity
- Possible transformations:
 - Geometric transformations (rotation, flipping, rescaling ...)
 - Photometric adjustments (brightness, contrast ...)